

Creation of knowledge graph from free text using Neo4j and Stanza: An application in machine translation from English to Spanish

Theodoros Mavrovitis
Hellenic Open University
Patras, Greece
tmavrovitis@hotmail.com

Ioannis Refanidis
University of Macedonia
Thessaloniki, Greece
yrefanid@uom.edu.gr

Abstract

This paper studies the creation of a knowledge graph from free text using tools of Natural Language processing and machine learning. The paper focuses on different aspects of NLP and knowledge graph creation, giving a detailed description of the methodologies and tools used. We present key concepts of NLP such as NLU (Natural Language Understanding) and NLG (Natural language generation) as well as the different tools used such as Stanford NLP (Stanza). We present a practical example of creating a knowledge graph with Neo4j, from a 5 sentences text, and we go through all our used techniques such as the design of the entities and relationships, coding classes and algorithms etc. We use the example text of 5 sentences to create the knowledge graph, and also to propose an modern application of translation our 5 sentences text from English to Spanish.

CCS Concepts

• : Computing technologies; • Artificial Intelligence; • Natural Language Processing; • Information Systems; • Data management systems; • Database design and models; • Graph-based databased models;

Keywords

Machine Learning, Neo4j, Entity Recognition, Stanza,NLP

ACM Reference Format:

Theodoros Mavrovitis and Ioannis Refanidis. 2024. Creation of knowledge graph from free text using Neo4j and Stanza: An application in machine translation from English to Spanish. In *28th Pan-Hellenic Conference on Progress in Computing and Informatics (PCI 2024)*, December 13–15, 2024, Egaleo, Greece. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3716554.3716624>

1 Introduction

In the field of Computer Science, artificial intelligence and machine learning we find Natural Language processing (NLP) which focuses on the text translation and understanding from computers to human language. With NLP , machines can process human language

and are generating outcomes such as machine translation [3], sentiment analysis [10] and chatbots. Documents, articles, reports, emails etc. and all of unstructured free text can be processed so important information can be extracted to generate meaningful insights. One way to do so, is to convert the unstructured text to structured data – such as knowledge graphs – which can help us analyze, query, and visualize all the extracted information. One of the most important applications of NLP is language translation, which allows the conversion of text from one human language to another, facilitating global communication [3]. Other applications of NLP in our day to day can be chatbots and virtual assistants in customer service but also in code development. In parallel, with the evolution of knowledge graphs for representing and visualizing information we can structure the integration and retrieval of complex information in away that is more visual and informative. After all, a knowledge graph is structured representation of knowledge in the form of a graph with entities and relationships along with all the properties that we can extract from the free text.

This paper is focusing on the connection of knowledge graphs and NLP, especially in the creation of a knowledge graph from a free text using modern NLP techniques and tools like stanza. Our goal is to leverage from both technologies to develop new methods and attributes of extracted free text to a knowledge graph [7]. We really to tried to enhance the text extraction and information, with the goal to advance the contribution of areas as knowledge retrieval , data analysis and language understanding. We propose a workflow that converts a sample unstructured text to a Neo4j knowledge graph, by using entity extraction, relationship properties knowledge saving and graph database construction [4].

The rest of the paper is structured as follows: Section 2 presents the related background. Section 3 presents the proposed methodology, whereas Section 4 presents the experimental results. Section 5 paper discusses the overall approach whereas Section 6 concludes the paper and poses future directions.

2 Background

Previous work in the field of text analysis and therefore knowledge graph creation was primarily focused on semi-automated techniques that required partial human involvement for data curation and manual annotation which limited the scalability of all those models [2]. A human supervision often included tasks such as entity identification, relationship between words and sentences and making sure the correct relationships were created. As NLP evolved through the years the human involvement became less needed, as several parts of the knowledge extraction from free text can be



This work is licensed under a Creative Commons Attribution International 4.0 License.

PCI 2024, Egaleo, Greece

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1317-0/2024/12

<https://doi.org/10.1145/3716554.3716624>

Table 1: Model Comparison

Model	Strengths	Weaknesses	Best Use Cases in Knowledge Graphs
GPT	Generative, handles complex contexts, adaptable with prompts	High computational cost, may need fine-tuning	Generating triples, domain-specific text parsing
T5	Text-to-text flexibility, high accuracy in structured text generation	Requires task-specific fine-tuning	End-to-end triple extraction, Cypher query output
BERT	Contextual understanding, excels at comprehension tasks like NER, parsing	Not generative, lacks direct triple output	Entity recognition, relationship disambiguation

automated, although still some human supervision is required to have high-quality results.

There were several approaches during the years such as OpenIE (Open Information Extraction) that have been very important in extracting entities and relationships from text, but there is a lack of integration with graph databases like Neo4j [5]. Stanford’s OpenIE system has proven very effective in extracting triples from sentences (subject-verb-object), without human supervision, which can be used to form the building blocks of knowledge graphs, yet the challenge is still lying in the integration with graph databases like Neo4j [11] and build Knowledge graphs. One of the biggest limitations is that systems like Stanford’s OpenIE and other knowledge extractions systems, is the lack of native support for graph representations. Therefore, additional steps are needed to convert all the extracted information from the free text such entities and relationships to graph compatible format, which makes the whole workflow rather complex. OpenIE approach generates open-domain extractions that might be easy to use with specific-domain applications unless further procession is conducted [5].

With recent advancements in NLP, the Transformer-based models came to play such as GPT, T5 and BERT. With the evolution of those models, we minimized even more the need for manual effort in extracting entities and relationships from free text. This model leverage the Transformer architecture’s ability to capture dependencies and contextual relationships, even in longer range than 2 words, which make them extremely accurate for knowledge graph construction tasks. For example, GPT is great in generating triples (subject-verb-object) and domain-specific parsing, while T5 can directly produce outputs like Cypher queries for Neo4j. BERT, on the other hand, is extremely strong in in-depth text comprehension, such as syntactic analysis.

Table 1 below compares these models, highlighting their strengths, weaknesses, and suitability for knowledge graph construction:

Furthermore, tools like Regex were created and there was an even bigger advance in NLP techniques. Regex is a pattern-based extraction tool, which can be used to identify predefined phrases (entities-relationships) [6]. It is very efficient and simple tool, but it lacks context awareness which can be limited in understanding complex sentence structures. Often it is combined with Transformer-based models like BERT or T5, to balance efficiency and context awareness depth. For example, Regex is used for preliminary tagging of key entities and by using BERT or T5 can be refined and contextualized. This hybrid approach combines Regex speed and efficiency

with the more advanced parsing and relationship extraction of a Transformer based model.

Summarizing, while earlier approaches using Regex or OpenIE laid the groundwork for knowledge graph construction, modern Transformer-based models provide a more scalable, accurate and adaptable framework to address the challenges of extracting knowledge from unstructured text. There is a big advancement in reducing manual intervention and enabling direct integration with graph databases like Neo4j, significantly improving efficiency and precision of knowledge graph construction techniques.

3 Methodology

In our methodology we used tools like Stanza from Stanford NLP Group for text processing and Neo4j application for knowledge graph construction and manipulation [8], [1]. Stanza is an advanced and modern natural language processing library that is developed by Stanford NLP Group and can extract entities, lemmas, Part-of-Speech tagging, dependency parsing and constituency parsing as well as sentiments from tokens (words). We selected Stanza because of its performance and integration with many models and systems [3], as well as its support of 70 languages. Neo4j, is a graph database software designed to create, store and manage large amounts of interconnected data for creating and querying knowledge graphs [1],[4]. We combined and integrated these two tools, with the goal to create a pipeline that converts unstructured free text to a structured quarriable graph with entities relationships and their properties.

The process starts with coding the necessary classes and algorithms in Python, to tokenize and break down the unstructured text to chunks and extract all the information needed to assign to our nodes and relationships. So, the first step of the process is to design a schema of the representation of entities(nodes) and their equivalent relationships (edges). All the words and tokens will be represented as nodes, and all the syntactic relationships that are found will be represented as edges between the nodes. Therefore, we start with the implementation of Python scripts that use Stanza library to process the input text. Those scripts basically use Stanza’s capabilities like tokenization, lemmatization, part-of-speech tagging and syntactic dependency parsing. All the above operations are of outmost importance while we identify the structure of the text, which later will help us build our knowledge graph the as accurate as possible.

The way we used to extract properties was to tokenize, lemmatize and tag the words part-of-speech, or as it is called widely natural language preprocessing. In detail, tokenization breaks down our

text to sentences and words, called tokens, which are our building blocks for analysis. Lemmatization converts each token or word to its base or dictionary form (lemma) in our effort to extract knowledge and not specific word forms. For example, the variation “flicker”, “flickered” will have the same lemma. In a similar way, part-of-speech (POS) tagging assigns a grammatical category to each word such as noun, verb, adjective etc. which is very important to understanding the syntactic structure of the sentence. Finally, part of natural language preprocessing is the removal of “stop” words like “and” and “or” that do not provide and value in the meaning of the text.

Having extracted all the properties such as syntactic relations, sentiment, Named Entity Recognition (NER), word dependency relationships, lemma etc. [8], [7] we are attaching them to each token that will form our future node or entity on the knowledge graph, and eventually will be part of the properties of each node. For example, the syntactic relationships between words, such as subject-verb-object triplet are used to form the correct relationships on our graph. We are not doing only triplets, but we take it a step further for more complex syntactic relationships on our text. Named Entity Recognition (NER), a key feature in NLP, is used to identify and categorize nouns like people names, places and organizations that exist in our text. Additionally, we can extract information about the sentiment of each token, which can be useful in understanding the feeling or the sentiment of the token or sentence, very useful in decision-making systems [10]. Also, the word dependency that can be extracted with the help of Stanza play a very important role the graph creation as it helps create the correct relationships (edges) in the graph for each sentence.

As we can see in Figure 1 all entities (nodes) are created in the lemma form of each word, where all the forms of the word are saved as well. When constructing the graph, we store each entity in its lemma form to ensure consistency across different sentence constructions. For example, the lemma for “returns,” “returned,” and “returning” would all have been “return”, making sure that we create only one node for all those variations in the graph. Furthermore, our graph also stores on each node a breakdown of the word’s form such as Tense, Number and Person if the word is verb, or only number if the word is noun, to maintain information of the actual form of the word, even when we create lemma nodes. Storing both properties allows us to perform queries on both the lemma of the word but also on its specific form within the context of the sentence.

In all relationships of the entities, the current form of the word is saved as found in the text, as well as the “features” such as tense, person, and number, which can provide all the information needed for the application created to return the text in a different language. All the relationships created also storing all the information we extract with Stanza such tense, number and person as well as POS tagging and dependency tagging. For example, when a verb is connected to its subject and object, the actual form of the verb broken down to tense (past, present, future), person (first, second, third) and the number (singular or plural) are stored as properties in the relationship of the verb with the sentence. This is very useful on our translation application where we are searching for the exact information to return the word in a different language, in our case in Spanish.

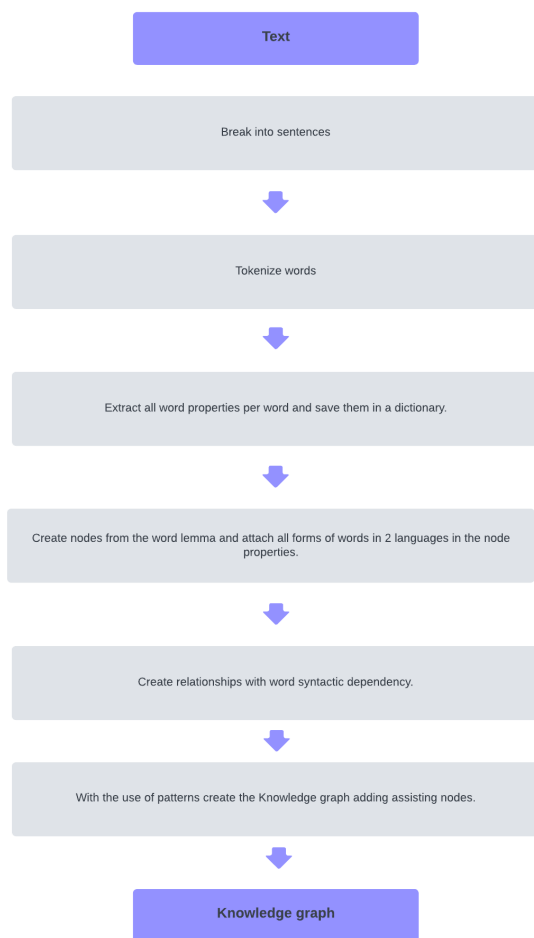


Figure 1: Summarized methodology

At this point, after relying on automatic NLP tool like Stanza, we introduce the use of patterns and rules to have a better control on the relationship creation between our nodes and to refine our extraction process when creating the knowledge graph. Those patterns were manually crafted based on the dependency parsing and part-of-speech tags that was assigned by Stanza in the first step, allowing us to control how the graph is created. For example, patterns are used to ensure that the verbs are connected to the sentence node, but the adjective will be connected to the noun node and not to the sentence node directly. This pattern or rule-based methodology improves the accuracy of the graph when visualizing knowledge and helps avoiding all nodes connected between them or all nodes connecting to the sentence node (poor visualization).

In other words, patterns are manually created based on the syntactic dependencies of the words as extracted from Stanza and used to map relationships between the entities. The patterns used in this study are directly derived from the syntactic dependencies identified by Stanza [8] and are applied to a specific vocabulary for the words and sentences we used as part of our study. They

describe the connection or relation between the words in a sentence (i.e. which word is the subject of the verb or which word is the compound of a noun) and we are using them to reflect the actual structure of the sentence in our graph, rather than simply relying on the surface-level word occurrences and connection to the sentence. This allows much better visualization of the graph, closer to the actual meaning of each sentence which can be very powerful tool for information retrieval and analysis.

Therefore, the use of patterns plays a very important role in the control of the accuracy of knowledge graph as well as the relationships of the words. By using syntactic patterns, we can ensure a higher level of control over the knowledge graph construction process which is particularly important and useful in cases where words may appear in different syntactic attributes in different sentences. For example, the same word may act as a subject in one sentence and as an object in another sentence. There are also words that can act as verbs in one sentence and as adpositions (ADP) in another sentence. (i.e. the word “like”).

A pattern is created for each sentence, and all the information gathered by Stanza is adjusted and verified before creating the knowledge graph for the specific sentence. Then for each sentence in our text, a specific pattern is created to map the syntactic relations in each sentence based on the dependencies extracted by Stanza [8]. Before we create the graph, we validate and verify the information extracted by Stanza and we ensure it aligns with our patterns per each sentence. This validation step is crucial for the quality of the graph, as it allows us to correct any potential error or inconsistencies in the automatically extracted information.

Having said the above, by using the pattern method, we achieve greater accuracy in what NLP tools are providing in an automated way, and we have complete control over the outcome and the graph creation process, even when a word appears with different syntactic attributes or meaning in different sentences in the text. It is the combination therefore of patterns and NLP tools like Stanza, that allows us to take advantage of all technologies without compromising the result and visualization of the knowledge graph. Even if the same word is behaving differently knowledge wise in different sentences, our system can correctly capture the differences and represent them correctly in our graph giving us a flexibility of having different word scenarios without compromising the quality and visual representation of our knowledge graph.

Once the properties of each token (word) are extracted and verified, we are then mapping them to our knowledge graph as nodes and edges. Neo4j’s graph structure and the use of Cypher query language is ideal for representing the connections between text tokens, visualizing the words or tokens as nodes and syntactic relationships as edges connecting the nodes. This structure is extremely useful when we need to query the graph for information such as “find all the subjects linked to a specific verb” rather than depending on keyword searches. Cypher allows us queries like the above which makes the knowledge graph very powerful in terms of searching/querying for knowledge or information and not for specific keywords.

The final graph consists of nodes representing sentences, lemmas (e.g., Sentence node, Rootlemma: “be”) and edges representing relationships with the part-of-speech tagging between the nodes (e.g., Adj, conj, nsubj, etc.). We design the graph creating like this, in

our effort to capture knowledge and meaning of the sentences rather specific words. By creating a node with the lemma of the word “be” for example, we avoid creating too many nodes with all the different forms of the word “be” that we possibly encounter in our text. In the same way, each relationship between the words is labeled with the syntactic parsing and the relationship between the two words as extracted by Stanza, and the words are connected according to their meaning in the sentence and not purely as extracted by Stanza.

Using all the above techniques, the final knowledge graph is created after removing stop words and information that do not add in the meaning or knowledge of the sentence (i.e. words like “the”, “and”, “or” and other non-informative elements are removed). This ensures that the knowledge graph represents knowledge and meaning rather than specific words and specific relations which is making it both compact and highly informative providing users with a powerful tool for text analysis.

4 Results

The knowledge graph that we created from the example text of 5 sentences demonstrates the effectiveness of the methods used and applied as well as the power of combining automated NLP tools with minimal manual control of the outcome. The end graph can be seen in Figure 2. Through our approach, we have successfully transformed unstructured text into a structured queryable dataset in a knowledge graph that captures accurately the visualization of the meaning and knowledge of each sentence rather than just their words and the automated generated relationships. This is significant as it showcases that accurate knowledge graphs can be created, and knowledge can be queried even with complex syntactic structures and words with different attributes per sentence in the real world.

The knowledge graph includes nodes such as sentences, lemmas created directly from the NLP extraction, as well as assisting nodes that we created for better knowledge visualization such as composite objects and composite subjects. The relationship’s labels were created based on the syntactic dependencies between the words and the connections created according to our patterns or rules-based methodology for each sentence. This structure ensures that the graph captures the essential elements of the sentence to map knowledge, rather than collecting specific words and phrases. For the relationships, each of them is defined based on the syntactic dependency of the words and are created with the use of manual patterns, enabling queries that can extract meaningful information on the interaction of the entities far beyond a keyword-based search system.

The nodes of our graph are labelled as Lemma and represent the lemma of all words as returned automatically from Stanza. On the properties of each node, we save all the information that can be extracted such as word.id, word-lemma, word-text, word-pos (part-of-speech), the word syntactic dependency, the sentence id the word is contained in etc. We also include properties such as if the word is recognized as Named Entity and sentiment value as it is extracted by Stanza. In addition, as we see below, we attach as properties to each lemma node, all the forms of the identified word in both English and Spanish language. In other words, for each word and node created we are attaching all the forms (past,

present, future, gerund, singular, plural etc. in English and Spanish that we use later to return the word form needed in a different language. Having all this info attached in the nodes allows to be able to extract information using queries that are more generic and more meaningful and don't depend on the superficial variations that each word form may have. This also allows us to keep the knowledge graph more efficient as we create one node per word as a lemma and not all the forms of the words as encountered.

The relationships of our graph have been detected correctly by our system and are storing all the necessary information of the destination word (node) such as sentence-id, actual form of the word as encountered (tense, number, person) and other information. We then combine the information we have stored in the destination node with the information stored on the relationship (actual word form) to accurately return the destination word in the correct form of another language, making possible an application of translation between different languages. Controlling the relationships with patterns allows us to return correctly the text on a different language, not depending on the syntactic differences between languages.

To do so, we have created a restricted dictionary with all the words found in our sample text of five sentences, in all forms, both in English and Spanish. This bilingual dictionary that has been created manually, is mapping all our vocabulary in a python Json dictionary starting with the language, in our case English and Spanish, and then for each language we break down all the forms of all the words used. For example, the word "seeking" is represented as the gerund of the word "seek" in English, and the equivalent entry in Spanish is the gerund of the word "buscar" which is "buscando". In the same way, we can return all the words of each sentence from English to Spanish in their correct form.

We therefore used the dictionary to store all the forms of the words in the knowledge graph, demonstrating the practical application of converting free text to knowledge graph for language translation. So, we first create the knowledge graph from free unstructured text, and by adding the above-mentioned dictionary we then convert the knowledge graph back to text in a different language. That showcases that knowledge graphs and our methodology can be successfully used to translate between different languages and with the use of patterns in the syntactic dependencies we can control the order of the words in each sentence which might be different for each language. For example, in English the adjective usually precedes the noun but in Spanish almost always the adjective is placed after the noun. We have proven that we can accurately control these differences between human languages in translating free unstructured text.

Concluding, the translation of English text to Spanish querying a knowledge graph to extract all the information needed showcases the potential on our approach. Using our methodology, we have successfully demonstrated that we can use NLP and knowledge graphs to translate sentences between two languages and with the use of patterns to better control the outcome, both when creating the graph but also when we return the text in different language, so we can correct the syntactic differences between languages. In other words, we can control and return grammatically correct and semantically accurate translations as we have better control over the outcome with the use of patterns. The knowledge graph approach has shown versatility and better control of the knowledge

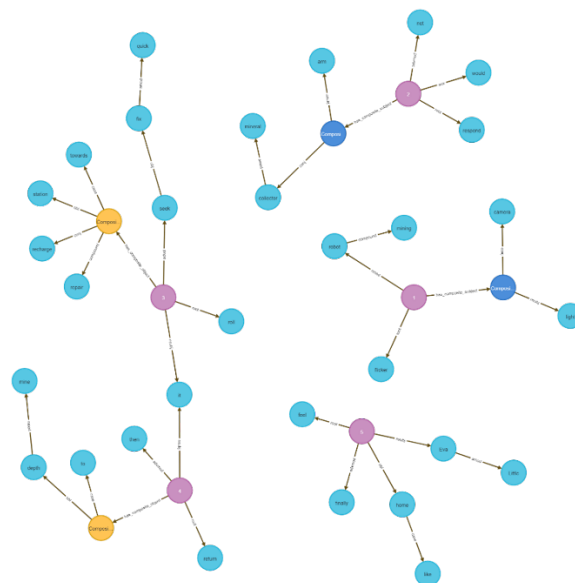


Figure 2: The knowledge graph

of the sentences, which can be extended to support not only translation but also other multilingual applications. The results indicate that our methodology can be a valuable tool when precise text understanding and translation is needed and can open new possibilities for various applications. For example, multilingual customer service applications, content management systems and knowledge sharing repositories can really benefit from our methodology.

The text we used to create the knowledge graph of Figure 2, is the following: *"The lights and cameras of the mining robot flickered. The arm and mineral collector wouldn't respond. It rolled towards the repair and recharge station seeking a quick fix. It then returned to the depths of the mine. Little Eva finally felt like home."*

5 Discussion

Our approach to create a knowledge graph with the use of Neo4j application is enabling us to better visualize the knowledge extracted from the free text, by representing nodes and relationships with all the tools that Neo4j allows. The core advantage is that the graph can be customized in terms of visuals, so it can be more intuitive depending on all the visualizations used (node colors, relationships thickness etc.). Customized visual representation of text can have many benefits as one can identify patterns in a big text corpus by examining the visual knowledge graph which can be easier than using the conventional methods [7] where relationships or even entities are hard to retrieve using traditional text-based representations and therefore makes it very difficult to examine, analyze and possibly encounter patterns in knowledge or specific meanings between words and sentences.

Our model allows for more efficient data retrieval and eventually better insight generation especially when dealing with free unstructured text. The key advantage is that we move away from keyword searches, like in traditional text databases, which might be imprecise or can miss important information and relationships that

are connected directly between them. In addition to enabling the execution of operations like employing shortest path algorithms to identify important entity relations or even identifying indirect relationships between entities, which are occasionally essential for producing insights, the knowledge graph approach can handle more complex queries that have the ability to target particular relationships and entities. It is those indirect relationships between entities, that are often overlooked in simple text searches [7] but they can play a key role in extracting relevant information. Therefore, we could navigate into complex relationships in domains like research, legal documents or business reports where uncovering hidden relationships can be crucial.

Furthermore, moving away from keyword-based searches to queries can significantly improve both the relevance and the precision of the queried information [9]. The traditional search methods usually rely on surface-level keyword matching, while querying a knowledge graph can leverage the actual construction of data [7]. In Neo4j for example, Cypher language is very powerful to construct and generate queries which allows users to perform not only precise searches based on relationships of specific entities (i.e. finding all instances where a certain entity acts as subject or as composite subject), but also we can take advantage of the knowledge graph ability to incorporate semantic understanding, which means that we can include in our queries related concepts like synonyms or semantically similar terms. The knowledge graph approach's effectiveness and user-friendliness are further benefits when using a structured knowledge graph instead of manually searching through text for every mention of a specific entity or its related relationships when exploring relationships between different entities. For example, with the knowledge graph, even if an entity appears in multiple contexts or with different syntactic roles (i.e. subject or object) all its relationships can still be visualized coherently, and moreover visual graph interfaces like Neo4j Desktop, allows interactive exploration where users can focus on specific nodes and relationships, facilitating even deeper analysis without the need of manual text processing [9]. Overall, knowledge graph is an invaluable tool for more complex queries and information retrieval as it drastically improves both the precision (by retrieving exactly what is needed) but also recall (retrieving all relevant results).

Our model had specific vocabulary as we saw above with specific five sentences that we created the patterns for, using the syntactic dependencies. Scaling the model might be a challenge as further manual work is required, although there might be many new ways to automate the pattern generation which need to be explored. While our model is successful both on creating the knowledge graph and translating the text from knowledge graph, some further research and work is needed to expand the model in bigger text corpora which currently presents as a notable challenge. Because each sentence needs to be analyzed to make sure the right patterns are found, creating the patterns for the relationship generation required manual labor. It is crucial to investigate automation techniques [4] for pattern creation without losing the manual control of graph creation, as the volume and complexity of unstructured text may make the manual approach difficult.

To overcome the challenge of scalability there are several potential solutions to investigate and chose the best one. The first one is the automation of pattern generation with machine learning

algorithms and rule-based systems [9], which might make possible to automatically detect recurring patterns and syntactic structures in free text, of course leveraging machine learning models trained on large text corpus, reducing the need of manual rule/pattern creation. The integration of such models with existing NLP tools like Stanza needs to be verified, to automatically identify new entities and relationships, adapting to variations in sentences structure without requiring human intervention [8]. The second possible solution would be to incorporate rule-based systems that will help automate the process of refining these patterns, always making sure of their precision and scalability. Active learning can be one of the techniques, where the system iteratively refines its patterns based on feedback so the automation process can be even more improved [9]. The third possible way is use semi-supervised learning and try to gradually reduce manual intervention. With this technique, a small amount of data can be used to train the model, which can then be applied new data, extracting new patterns of entities and relationships. Of course, as the system is exposed to more data it will have a better ability to generalize and create more patterns, allowing it to scale to larger text corpus while it maintains accuracy. Apart from reducing the workload, it could enable the model to adapt and use new vocabularies and sentence structures without constant human oversight [9].

Lastly, while scaling our model might present challenges, the potential and evolution of advanced NLP techniques might offer promising solutions. In the end, manual pattern creation with automated machine learning driven processes need to be combined, so our model can handle bigger and more complex text corpora, ensuring that the knowledge graph construction is both efficient and accurate as our successful model indicates.

6 Conclusions and future work

Concluding, this paper has successfully demonstrated an effecting way of integrating Natural Language Processing tools (Stanza) and knowledge graphs applications (Neo4j) to extract knowledge from free text, and furthermore return text from knowledge graph in a different language than the one creating the graph. We have developed a robust methodology to create knowledge graphs that accurately represent all the information that can be extracted from free text, in a controlled and accurate manner with the use of patterns. The practical application of this research such as the language translation, highlight the potential of knowledge graphs to improve various domains including information retrieval [12], data analysis and artificial intelligence [4]. We therefore prove in this study that a hybrid approach of machine learning and manual setup of patterns is the most accurate and efficient way to analyze free text.

The findings of our study emphasize the importance of combining NLP tools like Stanza and knowledge graphs to close the gap between free text and structured quarriable data, proven by the knowledge graphs generated in this paper which provide a comprehensive and intuitive representation of information, facilitating better understanding of text data. The approach we used improves the accuracy and efficiency of extracting information from text, but it also opens new ways for the development of intelligent systems and models, capable of more sophisticated text data analysis.

Although the results of our research are promising, several challenges and limitations were identified during this research, such as the scalability of our proposed methodology which may need to be further evaluated, particularly when dealing with large volumes of text corpora. Nevertheless, as NLP techniques improve, we have identified several ways to overcome the challenges, but they need to be investigated first.

Future work will be focusing on overcoming these challenges and adding new information on text analysis such as the identification of subsentences, chapters, more complex sentences and even entire books and technical documents. Furthermore, integrating additional data sources such as more structured databases and dictionaries can enrich the knowledge graphs and translate in even more languages. More tools like Regex can be added to reduce some of the manual work on the tokenization of words in a sentence, as well as Named Entity Recognition and Sentiment Analysis that we haven't used in our study [8]. Using our method, we can extract the latest information and attach it to our knowledge graph, which can then be used to query Named Entities and sentiments.

In addition, we have proven that the potential of knowledge graphs in supporting multilingual applications is very high, but also further investigation needs to be done. With knowledge graphs entering the multilingual space there can be a huge enhancement in the capabilities of NLP systems, with the different language syntactic rules can be addressed with the use of patterns, more and more languages can be added.

Summarizing, this paper lays the groundwork for future advancements in the integration of NLP tools like Stanza and knowledge graphs, by using a semi-automatic methodology to create knowledge graphs from free text. The techniques and methodologies presented and used have the potential to drive progress in space of text analysis, offering alternative ways to extract more text information and visualize in a structured data graph. [7]. Continuous research and development in this area with focus on integration

of those systems will lead to more sophisticated and impactful applications and will also help in bridging the gap between human language and machine understanding, processing and visualization. Finally, this paper demonstrates that the use of a hybrid model between machine learning and manual pattern creation can be the most efficient and accurate way to extract information from free text with the help of NLP techniques and then query and visualize the information with the creation of knowledge graph [3].

References

- [1] Angles, R., & Gutierrez, C. (2008). *Survey of graph database models*. *ACM Computing Surveys*, 40(1), 1–39. <https://doi.org/10.1145/1322432.1322433>
- [2] Auer, S., Bizer, C., Kobilarov, G., Lehmann, J., Cyganiak, R., & Ives, Z. (2007). *DBpedia: A nucleus for a web of open data*. *The Semantic Web*, 4825, 722–735. https://doi.org/10.1007/978-3-540-76298-0_52
- [3] Bahdanau, D., Cho, K., & Bengio, Y. (2014). *Neural machine translation by jointly learning to align and translate*. *arXiv preprint*. <https://arxiv.org/abs/1409.0473>
- [4] Barrasa, J., Hodler, A. E., & Webber, J. (2023). *Knowledge graphs: Data in context for responsive businesses*. O'Reilly Media.
- [5] Bollacker, K., Evans, C., Paritosh, P., Sturge, T., & Taylor, J. (2008). Freebase: A collaboratively created graph database for structuring human knowledge. *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 1247–1250. <https://doi.org/10.1145/1376616.1376746>
- [6] Richardson, M., & Domingos, P. (2002). *Markov logic networks*. *Machine Learning*, 62(1–2), 107–136. <https://doi.org/10.1023/A:1020328023373>
- [7] Ehrlinger, L., & Wöß, W. (2016). *Towards a definition of knowledge graphs*. In *Proceedings of SEMANTiCS 2016 (Posters, Demos, and SuCESS Tracks)*. <http://ceur-ws.org/Vol-1695/paper4.pdf>
- [8] Manning, C. D., Surdeanu, M., Bauer, J., Finkel, J., Bethard, S. J., & McClosky, D. (2014). *The Stanford CoreNLP natural language processing toolkit*. *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, 55–60. <https://doi.org/10.3115/v1/P14-5010>
- [9] Paulheim, H. (2017). *Knowledge graph refinement: A survey of approaches and evaluation methods*. *Semantic Web*, 8(3), 489–508. <https://doi.org/10.3233/SW-160218>
- [10] Poria, S., Cambria, E., Bajpai, R., & Hussain, A. (2017). *A review of affective computing: From unimodal analysis to multimodal fusion*. *Information Fusion*, 37, 98–125. <https://doi.org/10.1016/j.inffus.2017.02.003>
- [11] Robinson, I., Webber, J., & Eifrem, E. (2015). *Graph databases: New opportunities for connected data* (2nd ed.). O'Reilly Media.
- [12] Singhal, A. (2012). *Introducing the Knowledge Graph: Things, not strings*. Google Official Blog. Retrieved from <https://blog.google/products/search/introducing-knowledge-graph-things-not/>